
Configuring a Test Environment

Testbench Configuration

Topic Overview

Introduction

One of the key tenets of designing reusable testbenches is to make testbenches as configurable as possible. Doing this means that the testbench and its constituent parts can easily be reused and quickly modified (i.e. reconfigured).

In a testbench, there are any number of values that you might normally write as literals: values such as for-loop limits, string names, randomization weights, other constraint expression values and coverage bin values. These values can be represented by SystemVerilog variables, which can be set (and changed) at run-time, or SystemVerilog parameters, which must be set at elaboration time. Because of the flexibility they offer, variables organized into configuration objects and accessed using the `uvm_config_db` API should always be used where possible.

On the other hand, aspects like bus widths must be fixed at elaboration time, and so cannot be implemented via dynamic configuration objects. There are a number of articles on handling static parameters in UVM:

- The Parameterized Tests article shows how to use parameterized tests with the UVM factory.
- The Parameters Package article shows how to centralize the parameters shared between HDL/DUT and HVL/TB domains.
- The Parameters and Reuse article shows how to pass large sets of parameters down through the `uvm_component` hierarchy.

Configuration Objects

Configuration objects are an efficient, reusable means for organizing configuration variables. In a typical testbench there will generally be several configuration objects, each tied to a component. A configuration object is created as a subclass of `uvm_object` to encapsulate all related configuration variables for a given branch of the testbench structural hierarchy. There may also be a single, additional configuration object to hold global configuration variables. Each of the configuration variables within a configuration object may be declared as `rand` and consequently the configuration object may be randomized.

The UVM configuration database is effective in handling the scope and storage of user defined configuration objects. Below is the code for a typical agent configuration object. It has virtual interfaces to point to the driver and monitor BFM interfaces associated with the agent, and a number of variables to describe and control these BFM.

```
// configuration class
class wb_config extends uvm_object;
    `uvm_object_utils( wb_config );

// Configuration Parameters
    virtual wb_m_bus_driver_bfm wb_drv_bfm; // virtual driver BFM
    virtual wb_bus_monitor_bfm wb_mon_bfm; // virtual monitor BFM

    int m_wb_id; // Wishbone bus ID
    int m_wb_master_id; // Wishbone bus master id for
```

```

wishbone agent
  int m_mac_id; // id of MAC WB master

  int unsigned m_mac_wb_base_addr; // Wishbone base address of MAC
  bit [47:0] m_mac_eth_addr; // Ethernet address of MAC
  bit [47:0] m_tb_eth_addr; // Ethernet address of testbench
  for sends/receives

  int m_mem_slave_size; // Size of slave memory in bytes
  int unsigned m_s_mem_wb_base_addr; // base address of wb memory for
MAC frame buffers
  int m_mem_slave_wb_id; // Wishbone ID of slave memory
  int m_wb_verbosity; // verbosity level for wishbone
messages

  function new( string name = "" );
    super.new( name );
  endfunction

endclass

```

Using a Configuration Object

Any component that utilizes a configuration object should perform the following steps:

- Retrieve its configuration if its config object handle is null (i.e. not already set externally)
- Create its internal structure and define its behavior based on the configuration
- Configure its child (sub-)components

The test component, as the top-level component, gets its configuration values from either a test parameter package or from the UVM configuration database (e.g. for a virtual interface handle). It then sets test-specific configuration variables for components in the environment.

```

class test_mac_simple_duplex extends uvm_test;
  ...

  wb_config wb_config_0; // config object for WISHBONE BUS

  function void set_wishbone_config_params();
    // set configuration info
    // NOTE The MAC is WISHBONE slave 0, mem_slave_0 is WISHBONE
slave 1
    // MAC is WISHBONE master 0, wb_master is WISHBONE master 1
    wb_config_0 = new();

    if (!uvm_config_db #(virtual wb_m_bus_driver_bfm)::get(this, "",
"WB_DRV_BFM", wb_config_0.wb_drv_bfm))
      `uvm_fatal(...)
    if (!uvm_config_db #(virtual wb_bus_monitor_bfm)::get(this, "",
"WB_MON_BFM", wb_config_0.wb_mon_bfm))

```

```

        `uvm_fatal(...)

wb_config_0.m_wb_id = 0; // WISHBONE 0
wb_config_0.m_mac_id = 0; // the ID of the MAC master
wb_config_0.m_mac_eth_addr = 48'h000BC0D0EF00;
wb_config_0.m_mac_wb_base_addr = 32'h00100000;
wb_config_0.m_wb_master_id = 1; // the ID of the wb master
wb_config_0.m_tb_eth_addr = 48'h000203040506;
wb_config_0.m_s_mem_wb_base_addr = 32'h00000000;
wb_config_0.m_mem_slave_size = 32'h00100000; // 1 Mbyte
wb_config_0.m_mem_slave_wb_id = 0; // the ID of slave mem
wb_config_0.m_wb_verbosity = 350;

    uvm_config_db #(wb_config)::set(this, "*", "wb_config",
wb_config_0);
endfunction
...

function void build_phase(uvm_phase);
    set_wishbone_config_params();
    ...
endfunction
...

endclass

```

Notice above that ``uvm_fatal()` is used if a virtual interface is not found in the UVM configuration database. This will stop the test immediately with the given message passed to the ``uvm_fatal()` call. Optionally, these ``uvm_fatal()` messages could be converted to ``uvm_error()` messages to allow more of the testbench to run before being halted.

The components that use a configuration object either have the configuration object passed to it directly or retrieve it by using `uvm_config_db::get`. In this example, the driver gets the virtual interface handle, ID, and verbosity from the configuration object. Note that BFM's are not `uvm_components`. Therefore, the driver and/or monitor proxy may also need to pass pertinent configuration data to the BFM through a function call. A single function call generally suffices to pass this data all at once, minimizing any call overhead.

```

class wb_m_bus_driver extends uvm_driver #(wb_txn, wb_txn);
    ...

    local virtual wb_m_bus_driver_bfm m_bfm; // Virtual Interface

    wb_config m_cfg;

    function void build_phase( uvm_phase phase );

        if (m_config == null)
            if ( !uvm_config_db #( wb_config )::get( this , "" , "wb_config" ,
m_cfg ) ) begin
                `uvm_fatal(...)
            end
        end
    endfunction
endclass

```

```

        end
        m_bfm = m_cfg.wb_drv_bfm; // set local virtual if property
        ...
    endfunction

    function void connect_phase( uvm_phase phase );
        super.connect_phase( phase );
        m_bfm.set_m_id(m_config.m_wb_master_id); //Set config value in BFM
    endfunction

    function void end_of_elaboration();
        set_report_verbosity_level_hier(m_config.m_wb_wb_verbosity);
    endfunction

    ...

endclass

```

Configuring sequences

There is a separate article on configuring sequences.

Configuring DUT Connections

Setting up HDL-to-Testbench connections is a kind of configuration activity that is always necessary. A SystemVerilog module (typically the HDL side top level module or sometimes a finer level of module encapsulation) must add virtual interfaces as applicable into the configuration space. On the HVL testbench side the test component retrieves the pertinent virtual interface handles from the UVM config database and applies them to appropriate configuration objects:

```

class test_mac_simple_duplex extends uvm_test;
    ...

    function void set_wishbone_config_params();
        wb_config_0 = new();

        // Get the virtual interface handle that was set in the top module
    or protocol module
        if (!uvm_config_db #(virtual wb_m_bus_driver_bfm)::get(this, "",
"WB_DRV_BFM", wb_config_0.wb_drv_bfm))
            `uvm_fatal(...)
        if (!uvm_config_db #(virtual wb_bus_monitor_bfm)::get(this, "",
"WB_MON_BFM", wb_config_0.wb_mon_bfm))
            `uvm_fatal(...)
        ...
        uvm_config_db #( wb_config )::set(this, "*", "wb_config",
wb_config_0, 0); // put in config
    endfunction

    ...

```

```
endclass
```

Configuring Sequences

A Configurable Sequence

The most generic way to configure a sequence is to use its full hierarchical name by default, but allow any other name to be set instead as required:

```
class my_bus_seq extends uvm_sequence #( my_bus_sequence_item );
    string scope_name = "";

    task body();
        my_bus_config m_config;

        if( scope_name == "" ) begin
            scope_name = get_full_name(); // this is {
sequencer.get_full_name() , get_name() }
        end

        if( !uvm_config_db #( my_bus_config )::get( null , scope_name ,
"my_bus_config" , m_config ) ) begin
            `uvm_error(...)
        end
    endtask
endclass
```

Consider a sequence called "initialization_sequence" running on the sequencer "uvm_test_top.env.sub_env.agent1.sequencer". The scope_name in the code above is set by default to "uvm_test_top.env.sub_env.agent1.sequencer.initialization_sequence".

The most common use case for sequence configuration takes the agent's configuration object set for the agent and its constituent components (sequencer, driver, monitor, ...).

```
class sub_env extends uvm_env;
    ...

    function void build_phase( uvm_phase phase );
        ...
        my_bus_config agent1_config;
        ...
        uvm_config_db #( my_bus_config )::set( this , "agent1*" ,
"my_bus_config" , agent1_config );
        ...
    endfunction

    task main_phase( uvm_phase phase );
        my_bus_sequence seq =
```

```

my_bus_sequence::type_id::create("my_bus_sequence");

    seq.start( agent1.sequencer );
endtask
...
endclass

```

With the `sub_env` used as context, the `set()` call and the default `get()` call in the configurable sequence will match and result in the sequence having access to the agent's configuration object.

Per Sequence Configuration

The default full hierarchical scope name of the configurable sequence class above can be readily used to uniquely identify multiple sequence instances and hence subject them to different individual configurations. All that is required is that the given instance names of the sequences are different.

For example, the environment class might look like this:

```

class sub_env extends uvm_env;
...

function void build_phase( uvm_phase phase );
...
my_bus_config agent1_config, agent1_error_config;
...
agent1_config.enable_error_injection = 0;
agent1_error_config.enable_error_injection = 10;

// most sequences do not enable error injection
uvm_config_db #( my_bus_config )::set( this , "agent1*" ,
"my_bus_config" , agent1_config );

// sequences with "error" in their name will enable error injection
uvm_config_db #( my_bus_config )::set( this ,
"agent1.sequencer.error*" , "my_bus_config" , agent1_error_config );
...
endfunction

task main_phase( uvm_phase phase );
    my_bus_sequence normal_seq =
my_bus_sequence::type_id::create("normal_seq");
    my_bus_sequence error_seq =
my_bus_sequence::type_id::create("error_seq");

    normal_seq.start( agent1.sequencer );
    error_seq.start( agent1.sequencer );
endtask
...
endclass

```

Since the configurable sequence class uses the given sequence name to do a `config_db get()` call, the normal sequence will pick up the configuration object with error injection disabled, while the error sequence will pick up the error configuration object.

Ignoring the Component Hierarchy Altogether

It is also possible to completely ignore the component hierarchy for sequence configuration. The advantage would be that one can in effect define behavioral scopes that are intended only for configuring sequences, and keep these behavioral scopes completely separate from the component hierarchy. The configurable sequence described above will work in this scenario as well.

For example in a virtual sequence one might have:

```
class my_virtual_sequence extends uvm_sequence #(
  uvm_sequence_item_base );
...
task body();
    my_bus_sequence normal_seq =
my_bus_sequence::type_id::create("normal_seq");
    my_bus_sequence error_seq =
my_bus_sequence::type_id::create("error_seq");

    normal_seq.scope_name =
"sequences::my_bus_config.no_error_injection";
    error_seq.scope_name =
"sequences::my_bus_config.enable_error_injection";

    normal_seq.start( agent1.sequencer );
    error_seq.start( agent1.sequencer );
endtask
...
endclass
```

The downside to the added flexibility of this use model is that every sequence and component in the testbench must agree on the naming scheme, or at least be capable to deal with this kind of arbitrary naming scheme. As there is no longer guaranteed uniqueness of scope name, some reuse challenges may arise when transitioning from block to integration level testbench.

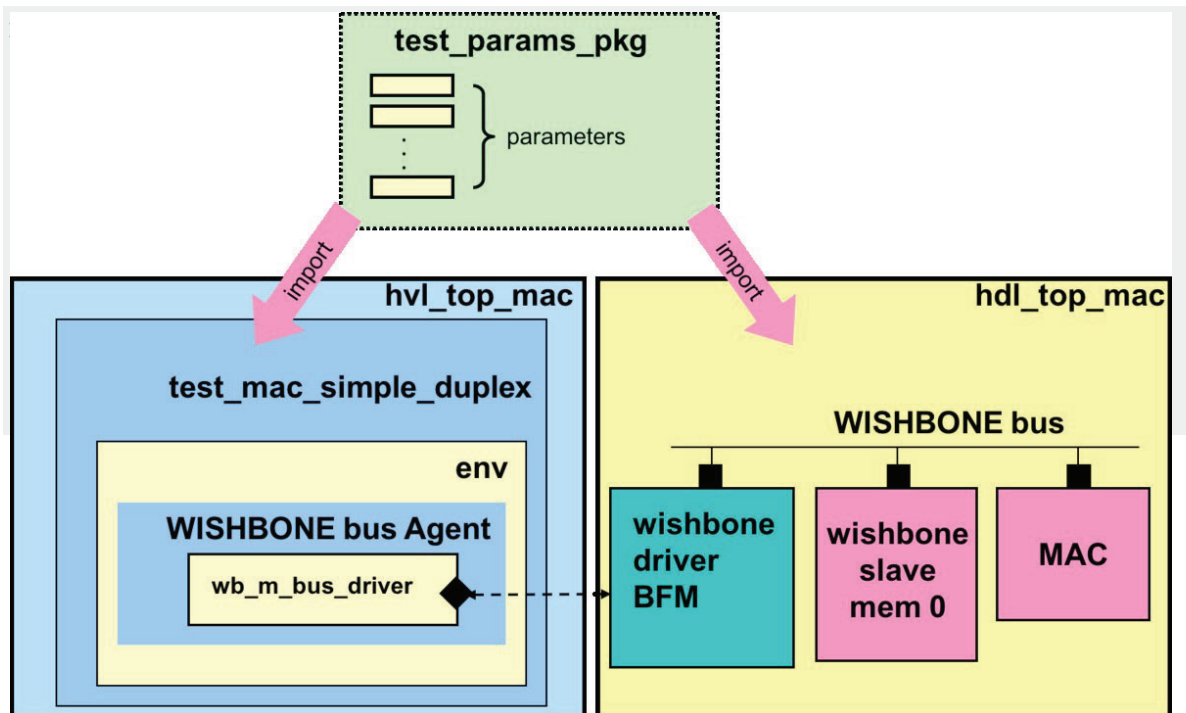
Using a Parameter Package

When a DUT or interface is parameterized, the parameter values are almost always used in the testbench as well. For this reason, these *common* parameters should not be specialized with direct literal values for your instance declarations. Define instead corresponding named parameters with associated values in a package to be shared by both the HDL/DUT side and testbench side of the environment. This greatly helps avoiding mistakes where a parameter value change would be made on one side but not the other, or where a test configuration parameter is some function of a DUT parameter and a miscalculation would result when making a change.

Note that this "shared package" need not be the place for all test parameters. Test parameters not applicable and used by the DUT can be specialized directly in the test. The shared parameters package should include only parameters shared between HDL/DUT and HVL/TB domains.

Example use of a Parameter Package

The WISHBONE example below involves two WISHBONE bus devices, slave memory and an Ethernet MAC (Media Access Controller). Parameters are put in a package *test_params_pkg* and used in instantiating the WISHBONE devices in the HDL top level module and in the test class on the testbench side.




```
// MAC WISHBONE parameters
parameter mac_m_wb_id = 0;      // WISHBONE bus master id of MAC
parameter mac_slave_wb_id = 1;  // WISHBONE bus slave id of MAC

endpackage
```

The usage of parameters `mem_slave_size` and `mem_slave_wb_id` in the HDL top module to instantiate the WISHBONE bus slave memory module is shown below. Note the import of the `test_params_pkg` in the `top_mac_hdl` module:

```
module hdl_top_mac;
    ...

    import test_params_pkg::*;

    // WISHBONE interface instance
    // Supports up to 8 masters and up to 8 slaves
    wishbone_master_bfm    wb_mstr_bfm(wb_bus_if);
    wishbone_bus_syscon_if wb_bus_if();

    //-----
    // WISHBONE 0, slave 0: 000000 - 0fffff
    // this is 1 Mbytes of memory
    wb_slave_mem #(mem_slave_size) wb_s_0 (
        // inputs
        .clk ( wb_bus_if.clk ),
        .rst ( wb_bus_if.rst ),
        .adr ( wb_bus_if.s_addr ),
        .din ( wb_bus_if.s_wdata ),
        .cyc ( wb_bus_if.s_cyc ),
        .stb ( wb_bus_if.s_stb[mem_slave_wb_id] ),
        .sel ( wb_bus_if.s_sel[3:0] ),
        .we  ( wb_bus_if.s_we ),
        // outputs
        .dout( wb_bus_if.s_rdata[mem_slave_wb_id] ),
        .ack ( wb_bus_if.s_ack[mem_slave_wb_id] ),
        .err ( wb_bus_if.s_err[mem_slave_wb_id] ),
        .rty ( wb_bus_if.s_rty[mem_slave_wb_id] )
    );

    ...
endmodule
```

Parameter usage in the test class of the testbench to set the configuration object values for the WISHBONE bus slave memory is shown below. Note that instead of the numeric literal `32'h00100000`, an expression involving the named DUT parameter `mem_slave_size` is used to assign the address value.

```
package tests_pkg;
    ...
```

```
import test_params_pkg::*;
...
`include "test_mac_simple_duplex.svh"
endpackage

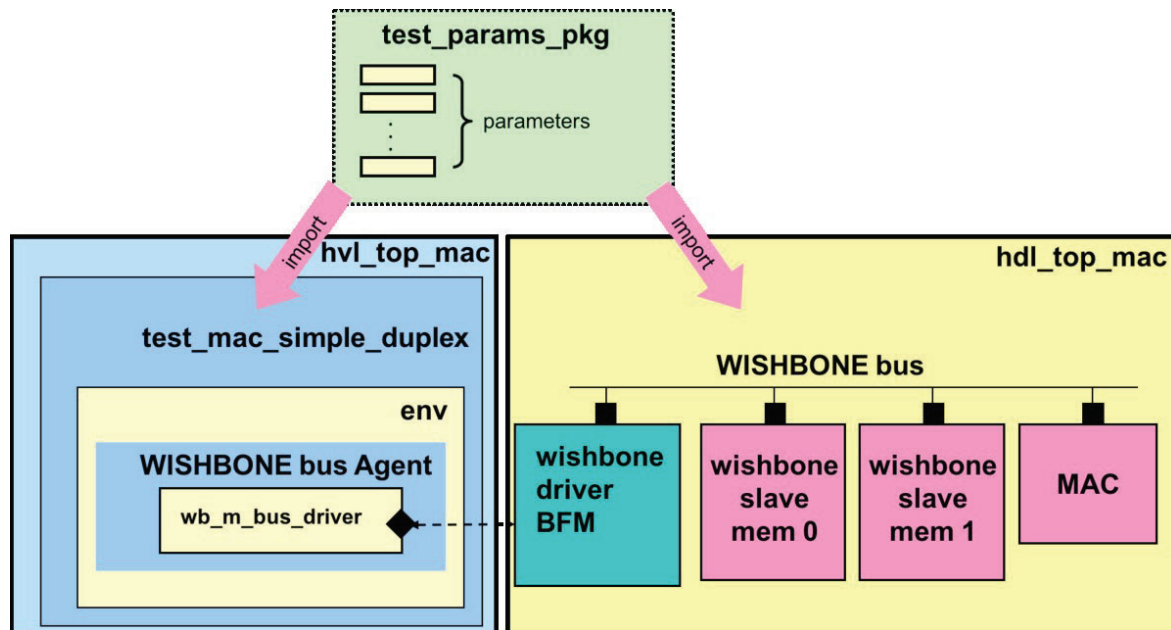
//-----
class test_mac_simple_duplex extends uvm_test;
...
    wb_config wb_config_0; // config object for WISHBONE BUS
    ...

    function void set_wishbone_config_params();
        //set configuration info
        wb_config_0 = new();

        wb_config_0.m_s_mem_wb_base_addr = mem_slave_wb_id *
slave_addr_space_sz; // base address of slave mem
        wb_config_0.m_mem_slave_size = 2**(mem_slave_size+2); // default
is 1 Mbyte
        wb_config_0.m_mem_slave_wb_id = mem_slave_wb_id; // WISHBONE bus
slave id of slave mem
        ...
    endfunction
    ...
endclass
```

Multiple Instances

In case of multiple instances of parameter sets an instance-mnemonic based naming convention can be used for distinguishing instances, or alternatively a parameterized class-based approach to distinguish the parameter sets by parameter specialization.



Note that now two instances of the Wishbone slave mem exist. Each has its own parameter specialization. This is handled through a parameterized class which specifies the parameters and their default values. The actual parameter values are then set for each instance by creating a specialization of the parameterized class using a typedef.

```
package test_params_pkg;
  import uvm_pkg::*;

  // WISHBONE general slave parameters
  parameter slave_addr_space_sz = 32'h00100000;

  // WISHBONE slave memory parameters
  class WISHBONE_SLAVE #(int mem_slave_size = 18, int mem_slave_wb_id =
0);
  endclass

  // Specializations for each slave memory instance
  typedef WISHBONE_SLAVE #(18, 0) WISHBONE_SLAVE_0;
  typedef WISHBONE_SLAVE #(18, 1) WISHBONE_SLAVE_1;

  // MAC WISHBONE parameters
  parameter mac_m_wb_id = 0;          // WISHBONE bus master id of MAC
  parameter mac_slave_wb_id = 2;      // WISHBONE bus slave id of MAC

endpackage
```

To use the parameters `mem_slave_size` and `mem_slave_wb_id` of the specialization `WISHBONE_SLAVE_0` or `WISHBONE_SLAVE_1` in the above code, use the following syntax: `name_of_specialization::parameter_name` as

illustrated below.

```

module hdl_top_mac;
    ...

    import test_params_pkg::*;

    // WISHBONE interface instance
    // Supports up to 8 masters and up to 8 slaves
    wishbone_master_bfm wb_mstr_bfm(wb_bus_if);
    wishbone_bus_syscon_if wb_bus_if();

    //-----
    // WISHBONE 0, slave 0: 000000 - 0fffff
    // this is 1 Mbytes of memory
    wb_slave_mem #(WISHBONE_SLAVE_0::mem_slave_size) wb_s_0 (
        // inputs
        .clk ( wb_bus_if.clk ),
        .rst ( wb_bus_if.rst ),
        .adr ( wb_bus_if.s_addr ),
        .din ( wb_bus_if.s_wdata ),
        .cyc ( wb_bus_if.s_cyc ),
        .stb ( wb_bus_if.s_stb  [WISHBONE_SLAVE_0::mem_slave_wb_id] ),
        .sel ( wb_bus_if.s_sel[3:0] ),
        .we  ( wb_bus_if.s_we  ),
        // outputs
        .dout( wb_bus_if.s_rdata[WISHBONE_SLAVE_0::mem_slave_wb_id] ),
        .ack ( wb_bus_if.s_ack  [WISHBONE_SLAVE_0::mem_slave_wb_id] ),
        .err ( wb_bus_if.s_err  [WISHBONE_SLAVE_0::mem_slave_wb_id] ),
        .rty ( wb_bus_if.s_rty  [WISHBONE_SLAVE_0::mem_slave_wb_id] )
    );

    ...
endmodule

```

Accessing Configuration Resources from a Sequence

Sequences often need access to testbench resources such as register models or configuration objects. This is best done using the `uvm_config_db` to retrieve a resource, as the first action of the `body()` method of a sequence base class that is extended by other sequences.

There are several ways in which the `uvm_config_db` can be used to access the resource:

- Access the resource using the `m_sequencer` handle

```
// Resource access using m_sequencer:
spi_env_config m_cfg;

task body();
    if(!uvm_config_db #(spi_env_config)::get(m_sequencer, "",
"spi_env_config", m_cfg)) begin
        `uvm_error("BODY", "spi_env_config config_db lookup failed")
    end
endtask: body
```

- Access the resource using the sequence's `get_full_name()` call

```
// Resource access using get_full_name():
spi_env_config m_cfg;

task body();
    if(!uvm_config_db #(spi_env_config)::get(null, get_full_name(),
"spi_env_config", m_cfg)) begin
        `uvm_error("BODY", "spi_env_config config_db lookup failed")
    end
endtask: body
```

- Access the resource using a scope string when the resource is not tied to the component hierarchy

```
// Resource access using pre-assigned lookup:
spi_env_config m_cfg;

task body();
    if(!uvm_config_db #(spi_env_config)::get(null, "SPI_ENV:",
"spi_env_config", m_cfg)) begin
        `uvm_error("BODY", "spi_env_config config_db lookup failed")
    end
endtask: body
```

The first two methods are mostly equivalent since they are both creating a scope string, based on the `m_sequencer` location in the testbench hierarchy or the sequence "pseudo" location in the testbench hierarchy, respectively. The small difference between these two methods is as follows. With the first method, `m_sequencer.get_full_name()` is called when `m_sequencer` is passed as the argument to the `get()` call, yielding the path to the sequencer in the testbench hierarchy. An example of this might be `"uvm_test_top.env.my_agent.sequencer"`. With the second method, `get_full_name()` is called on the sequence, not the sequencer. If the sequence was started on a sequencer (i.e. the

m_sequencer handle is not null), this get_full_name() call returns the path to the sequencer **with** the sequence name appended. An example of this might be "uvm_test_top.env.my_agent.sequencer.my_seq". This can be useful to target specific configuration information for a specific sequence running on a specific sequencer.

The third method relies on users agreeing to a naming convention for different configuration domains, which can work well within a particular project or workgroup but may cause reuse issues due to name clashes.

A full example of a base sequence implementation is shown below. A derived sequence must call the base sequence body() method to ensure that resource handles are set up before starting the stimulus process.

```
//  
// Sequence that needs to access a testbench resource via the  
configuration space  
//  
// Note that this is a base class; any class extending it must call  
super.body()  
// at the start of its body task to get set up  
//  
class register_base_seq extends uvm_sequence #(bus_seq_item);  
    `uvm_object_utils(register_base_seq)  
  
    // Handle for the actual sequencer to be used: bus_sequencer  
    BUS;  
  
    // Handle for the environment configuration object:  
    bus_env_config env_cfg;  
  
    // Handle for the register model  
    dut_reg_model RM;  
  
    function new(string name = "register_base_seq");  
        super.new(name);  
    endfunction  
  
    task body;  
        // Get the env configuration object - using get_full_name()  
        if(!uvm_config_db #(bus_env_config)::get(null,  
get_full_name(), "bus_config", env_cfg)) begin  
            `uvm_error("BODY", "Failed to find bus_env_config in the  
config_db")  
        end  
    end
```

```
// Assign a pointer to the register model which is
inside the env config object:
    RM =
    env_cfg.register_model;
    endtask: body
endclass: register_base_seq

//
// A derived sequence:
//
class initialization_seq extends register_base_seq;
    `uvm_object_utils(initialization_seq)

    task body;
        super.body(); // assign the resource handles
        ... // Sequence body code endtask:
    body
endclass: initialization_seq
```

Another example of using this technique is to access a virtual interface within a configuration object to wait for hardware events.

Macro Cost-Benefit Analysis

Macros can be useful to reduce repetitive typing of small pattern-like code segments, to hide implementation differences or limitations among the simulators from different vendors, or to make critical code segments less error-prone for reuse. Many of the macros in UVM meet these criteria, but not all. While the benefits of macros in general may be obvious and immediate, often the costs associated with their usage is not transparent and may well become problematic at a later point when code changes are increasingly obtrusive.

The topic of macro usage is explored in ample detail in the DVCon 2011 paper entitled OVM-UVM Macros-Costs vs Benefits DVCon11 Appnote (PDF) ^[1]:

- It examines the hidden costs incurred by some macros, including code bloat, low performance, and debug difficulty.
- It identifies which macros provide a good cost-benefit trade-off, and which do not.
- It shows how to replace high-cost macros with simple SystemVerilog code.

The recommendations are summarized as follows:

Macro Type	Description
<code>`uvm_*_utils</code>	Always use. These register the object or component with the UVM factory. While not a lot of code, registration can be hard to debug if not done correctly.
<code>`uvm_info` `uvm_warning` `uvm_error` `uvm_fatal`</code>	Always use. These can significantly improve performance over their function counterparts (e.g. <code>uvm_report_info</code>).
<code>`uvm_*_imp_decl</code>	OK to use. These enable a component to implement more than one instance of a TLM interface. Non-macro solutions don't provide significant advantage.
<code>`uvm_field_*</code>	Do not use. These inject lots of complex code that substantially decreases performance and hinders debug.
<code>`uvm_do_*</code>	Do not use. These unnecessarily obscure a simple API and are best replaced by a user-defined task, which affords far more flexibility.
<code>`uvm_sequence_utils` `uvm_sequencer_utils` other related macros</code>	Do not use. These macros built up a sequencer's sequence library and enable automatic starting of sequences, which is almost always the wrong thing to do.

References

- [1] <https://verificationacademy.com/resource/6646>